

아래는 **Gradio UI + vLLM** 서버를 사용하는 전형적인 구조와 예제입니다.
이 구조는 **GPU** 서버에서 **LLM**을 직접 서빙할 때 많이 사용됩니다.

핵심 구성

Browser



Gradio UI



vLLM Server (OpenAI compatible API)



LLM Model (GPU)

즉

Gradio = Frontend

vLLM = LLM inference server

1. vLLM이 무엇인가

vLLM 은 GPU에서 **LLM inference**를 매우 빠르게 수행하기 위한 서빙 엔진입니다.

특징

- PagedAttention memory management
- batching 자동화
- OpenAI API compatible
- HuggingFace 모델 바로 로딩

대표적으로 사용하는 모델

- Llama 3
- Mistral 7B

2. 전체 시스템 구조

Gradio + vLLM 구조

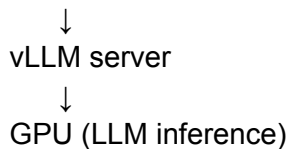
User Browser



Gradio Web UI



OpenAI compatible API



역할

구성	역할
Gradio	웹 UI
vLLM	LLM 서버
GPU	모델 실행

3. vLLM 서버 실행

먼저 GPU 서버에서 vLLM을 실행합니다.

설치

```
pip install vllm
```

모델 서버 실행

```
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Meta-Llama-3-8B-Instruct \
  --port 8000
```

서버가 실행되면

<http://localhost:8000>

OpenAI API와 동일한 endpoint가 생깁니다.

예

POST /v1/chat/completions

4. Gradio UI 코드

이제 Gradio가 **vLLM** 서버를 호출합니다.

예제 코드

```
import gradio as gr
import requests

VLLM_URL = "http://localhost:8000/v1/chat/completions"

def chat(message, history):

    messages = []

    for h in history:
        messages.append({"role": "user", "content": h[0]})
        messages.append({"role": "assistant", "content": h[1]})

    messages.append({"role": "user", "content": message})

    payload = {
        "model": "meta-llama/Meta-Llama-3-8B-Instruct",
        "messages": messages,
        "temperature": 0.7,
        "max_tokens": 512
    }

    response = requests.post(VLLM_URL, json=payload)

    result = response.json()

    answer = result["choices"][0]["message"]["content"]

    return answer

demo = gr.ChatInterface(chat)

demo.launch()
```

실행

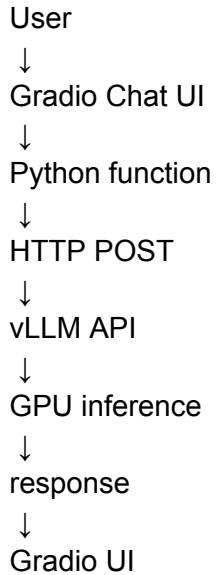
```
python app.py
```

브라우저

<http://localhost:7860>

5. 실행 흐름

사용자가 질문하면 다음 흐름이 발생합니다.



6. 실제 **request** 예시

Gradio가 보내는 HTTP request

POST /v1/chat/completions

payload

```
{  
  "model": "meta-llama/Meta-Llama-3-8B-Instruct",  
  "messages": [  
    {"role": "user", "content": "Explain transformers"}  
  ],  
  "temperature": 0.7  
}
```

vLLM 응답

```
{  
  "choices": [  
    {  
      "message": {  
        "role": "assistant",  
        "content": "Transformers are neural architectures..."  
      }  
    }  
  ]  
}
```

```
}  
]  
}
```

7. Gradio + vLLM 장점

이 구조가 인기 있는 이유

① OpenAI API 호환

즉 코드 변경 거의 없음

OpenAI API
→ vLLM API

② GPU utilization 높음

vLLM 특징

dynamic batching
KV cache reuse
PagedAttention

③ UI와 inference 분리

UI server
LLM server

독립 scaling 가능

8. 실제 연구용 아키텍처

연구실에서 많이 쓰는 구조

Browser
↓
Gradio UI
↓
Backend API (FastAPI)
↓

vLLM server



GPU cluster

추가 구성

Vector DB

Redis

RAG pipeline

9. RAG까지 포함한 구조

User



Gradio UI



FastAPI backend



Vector DB



Context retrieval



vLLM



Response

vector DB 예

- FAISS
- Qdrant

10. Streaming 출력 (LLM 스타일)

Gradio는 streaming도 지원합니다.

예

```
def chat(message, history):
```

```
    response = requests.post(
        VLLM_URL,
        json={
            "model": "meta-llama/Meta-Llama-3-8B-Instruct",
```

```
        "messages": [{"role": "user", "content": message}],
        "stream": True
    },
    stream=True
)
```

```
partial = ""
```

```
for line in response.iter_lines():
    if line:
        partial += line.decode()
        yield partial
```

이렇게 하면

ChatGPT처럼
토큰이
실시간 출력

됩니다.

11. 핵심 정리

Gradio + vLLM 구조

Frontend
Gradio

LLM Server
vLLM

Model
Llama / Mistral

동작

Gradio → HTTP → vLLM → GPU → response

장점

- OpenAI API 호환
- GPU 효율 매우 높음
- 로컬 LLM 서비스 구축 가능

추가로 중요한 것 (연구실 환경)

사용자 상황처럼 **T4 GPU** 서버 기반 **LLM** 교육/서비스라면 다음 구조가 가장 좋습니다.

Students
↓
Gradio UI
↓
Gateway API
↓
vLLM
↓
GPU server

여기에

RAG
Vector DB
Agent

를 붙이면 완전한 **LLM** 플랫폼이 됩니다.

원하시면 다음도 설명해 드리겠습니다.

- ❶ **vLLM + Gradio + RAG** 전체 코드 (300 lines)
- ❷ **T4 GPU 4대**에서 **vLLM** 서버 운영 구조
- ❸ **Streamlit vs Gradio** (LLM UI 관점)

특히 **Gradio vs Streamlit**은 **LLM** 서비스 설계에서 매우 중요한 선택입니다.

캡스톤 수업 환경(학생 PC + RTX 5070 수준 GPU)에서 **Gradio vs Streamlit**을 선택할 때는 단순 UI 프레임워크 비교보다 **LLM** 실험 **workflow** 관점에서 보는 것이 중요합니다.

특히 ****로컬 GPU inference (vLLM, Transformers)****와의 결합성이 핵심입니다.

아래 기준으로 분석하겠습니다.

1. 아키텍처
 2. LLM 실험 적합성
 3. GPU 로컬 실행
 4. UI 개발 난이도
 5. 확장성
 6. 교육/캡스톤 프로젝트 적합성
-

1. 기본 철학 차이

항목	Gradio	Streamlit
목적	ML 모델 데모	데이터 앱 / 대시보드
UI 방식	Component 기반	Script 기반
주 사용 분야	AI demo	Data dashboard
LLM 친화성	매우 높음	중간

- Gradio
- Streamlit

핵심 차이

Gradio = ML model interface
Streamlit = data application UI

2. 아키텍처 차이

Gradio

User
↓
Gradio UI
↓
Python function
↓
Model inference

모델 함수와 UI가 직접 연결

예

```
def generate(prompt):  
    return model.generate(prompt)
```

Streamlit

User

↓
Streamlit UI
↓
Script re-run
↓
Backend logic

Streamlit은 앱 전체가 재실행됩니다.

3. LLM 실험 관점

Gradio

LLM 데모 제작이 매우 쉽습니다.

예

```
import gradio as gr
from transformers import pipeline

pipe = pipeline("text-generation")

def chat(prompt):
    return pipe(prompt)[0]["generated_text"]

gr.Interface(chat, "text", "text").launch()
```

10줄이면 챗봇

Streamlit

같은 기능

```
import streamlit as st

prompt = st.text_input("Prompt")

if prompt:
    result = pipe(prompt)
    st.write(result)
```

가능하지만

state 관리
chat UI
streaming

을 직접 만들어야 합니다.

4. RTX 5070 로컬 실험

학생 PC 기준

GPU
RTX 5070 ($\approx$ 16GB)

가능한 모델

Llama 3 8B
Mistral 7B
Qwen 7B

- Llama 3
 - Mistral 7B
-

Gradio + 로컬 LLM

아키텍처

User
↓
Gradio
↓
Transformers / vLLM
↓
RTX 5070

코드

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch
import gradio as gr
```

```
model_name = "mistralai/Mistral-7B-Instruct"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

def chat(prompt):

    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")

    outputs = model.generate(**inputs, max_new_tokens=200)

    return tokenizer.decode(outputs[0])

gr.Interface(chat, "text", "text").launch()
```

학생 PC에서도 바로 실행됩니다.

Streamlit + 로컬 LLM

가능하지만

streaming
chat interface

구현이 더 복잡합니다.

5. UI 기능 비교

기능	Gradio	Streamlit
chat UI	built-in	직접 구현
streaming token	쉬움	약간 복잡
file upload	쉬움	쉬움
dashboard	약함	강함
layout	제한적	매우 좋음

6. 코드 복잡도

LLM 챗봇 기준

Gradio

10 ~ 40 lines

Streamlit

50 ~ 120 lines

7. RAG 프로젝트 적합성

캡스톤에서 흔한 프로젝트

PDF QA

Document chatbot

Company chatbot

Research assistant

구조

UI



Retriever



Vector DB



LLM

Gradio 구조

Gradio



RAG pipeline



FAISS

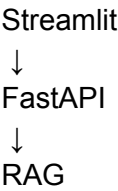


LLM

- FAISS

학생 구현 난이도 낮음.

Streamlit 구조



조금 더 웹 앱 구조에 가깝습니다.

8. 교육 관점 비교

항목	Gradio	Streamlit
학습 난이도	매우 낮음	낮음
LLM 실험	매우 좋음	보통
웹 앱 개발	보통	좋음
데이터 분석	약함	매우 좋음
캡스톤 적합성	매우 좋음	좋음

9. RTX 5070 캡스톤 추천 구조

학생 PC 기반



전체 구조

User
↓
Gradio UI
↓
RAG Pipeline
↓
FAISS
↓
LLM
↓
RTX 5070

이 구조는

OpenAI API 필요 없음
GPU 로컬 실행

가능합니다.

10. 실제 수업에서 가장 많이 쓰는 구조

캡스톤 수업에서는 보통 다음입니다.

UI
Gradio

LLM
Transformers

Vector DB
FAISS

Model
Mistral 7B

11. 결정 가이드

Gradio 선택

다음 목표라면

LLM demo
chatbot
RAG
multimodal AI

Streamlit 선택

다음 목표라면

AI dashboard
data analysis
business app

12. 캡스톤 수업 결론

학생 RTX 5070 PC 기준

추천

Gradio + Transformers + FAISS

이유

설치 쉬움
코드 짧음
LLM 친화적
GPU 바로 사용

✅ 한 줄 결론

LLM 캡스톤 수업 → Gradio
데이터 앱 프로젝트 → Streamlit

원하시면 제가 다음도 정리해 드릴 수 있습니다.

“RTX 5070 학생 PC 기준 LLM 캡스톤 프로젝트 10개”

이건 실제로 수업 운영할 때 난이도별 프로젝트 설계에 매우 도움이 됩니다.

아래에서는 **Gradio + Transformers + FAISS** 기반 **RAG** 챗봇을 예제로 설명하겠습니다.
특히 질문하신 **“FAISS를 설치 없이 사용할 수 있다”**는 말의 정확한 의미도 함께 설명합니다.

핵심 스택

UI

Gradio

LLM

Transformers

Vector Search

FAISS

- Gradio
 - Transformers
 - FAISS
-

1. 전체 구조 (RAG Pipeline)

Gradio + FAISS 구조는 다음과 같습니다.

User Question



Gradio UI



Embedding



FAISS Vector Search



Relevant Document



LLM (Transformers)



Answer

RAG 구조

Document → Embedding → Vector DB



Question → Embedding → Similarity Search



Context + Question → LLM

2. 필요한 라이브러리

기본 설치

```
pip install gradio transformers sentence-transformers faiss-cpu
```

학생 PC에서도 설치 가능합니다.

RTX 5070 같은 GPU에서는

faiss-cpu

만으로도 충분합니다.

3. 예제 데이터

예제 문서

```
documents = [  
    "Transformers are neural network architectures used in NLP.",  
    "RAG stands for Retrieval Augmented Generation.",  
    "FAISS is a library for efficient similarity search.",  
    "Large language models can answer questions using context."  
]
```

4. 문서 Embedding 생성

여기서는 embedding 모델을 사용합니다.

예

- all-MiniLM-L6-v2

코드

```
from sentence_transformers import SentenceTransformer  
  
model = SentenceTransformer("all-MiniLM-L6-v2")  
  
embeddings = model.encode(documents)
```

결과

```
embeddings.shape
```

(4, 384)

즉

문서 4개

vector dimension = 384

5. FAISS Index 생성

FAISS는 **vector index**를 생성합니다.

```
import faiss
import numpy as np

dimension = embeddings.shape[1]

index = faiss.IndexFlatL2(dimension)

index.add(np.array(embeddings))
```

구조

FAISS Index

```
├ vector1
├ vector2
├ vector3
└ vector4
```

6. 질문 검색

사용자 질문을 **embedding**으로 변환합니다.

```
query = "What is RAG?"

query_vector = model.encode([query])
```

FAISS 검색

```
D, I = index.search(query_vector, k=2)
```

결과

`l = [1,3]`

즉

`documents[1]`
`documents[3]`

가 가장 유사합니다.

7. LLM에 context 전달

검색된 문서를 context로 전달합니다.

```
context = "\n".join([documents[i] for i in l[0]])
```

```
prompt = f"""  
Answer using the following context.
```

```
Context:  
{context}
```

```
Question:  
{query}  
"""
```

8. Transformers LLM 실행

예

- Mistral 7B

간단한 모델 예제

```
from transformers import pipeline
```

```
generator = pipeline(  
    "text-generation",  
    model="distilgpt2"  
)
```

```
answer = generator(prompt, max_new_tokens=100)
```

```
print(answer[0]["generated_text"])
```

9. Gradio UI 연결

이제 전체를 Gradio와 연결합니다.

```
import gradio as gr

def rag_chat(question):

    query_vector = model.encode([question])

    D, I = index.search(query_vector, k=2)

    context = "\n".join([documents[i] for i in I[0]])

    prompt = f"""
    Context:
    {context}

    Question:
    {question}

    Answer:
    """

    result = generator(prompt, max_new_tokens=100)

    return result[0]["generated_text"]

gr.Interface(
    fn=rag_chat,
    inputs="text",
    outputs="text",
    title="Simple RAG Chatbot"
).launch()
```

실행

python app.py

브라우저

<http://localhost:7860>

10. 전체 코드 (정리)

전체 구조

```
import gradio as gr
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
from transformers import pipeline

documents = [
    "Transformers are neural networks used in NLP.",
    "RAG stands for Retrieval Augmented Generation.",
    "FAISS enables similarity search on vectors.",
    "LLMs answer questions using context."
]

embed_model = SentenceTransformer("all-MiniLM-L6-v2")

doc_vectors = embed_model.encode(documents)

index = faiss.IndexFlatL2(doc_vectors.shape[1])
index.add(np.array(doc_vectors))

generator = pipeline("text-generation", model="distilgpt2")

def rag_chat(question):

    query_vector = embed_model.encode([question])

    D, I = index.search(query_vector, k=2)

    context = "\n".join([documents[i] for i in I[0]])

    prompt = f"Context:{context}\nQuestion:{question}\nAnswer:"

    result = generator(prompt, max_new_tokens=100)

    return result[0]["generated_text"]

gr.Interface(fn=rag_chat, inputs="text", outputs="text").launch()
```

약 **80**줄입니다.

11. “FAISS 설치 없이 사용 가능” 의미

이 말은 정확히는 오해가 있습니다.

FAISS는 반드시 설치해야 합니다.

설치 방법

```
pip install faiss-cpu
```

또는

```
pip install faiss-gpu
```

왜 “설치 없이 가능”이라는 말이 나오나

이유는 다음 때문입니다.

❶ 일부 라이브러리가 내부적으로 **FAISS** 포함

예

- LangChain

LangChain

```
from langchain.vectorstores import FAISS
```

이 경우

LangChain이 FAISS wrapper 제공

하지만 실제로는

faiss library 필요

합니다.

❷ Colab 환경

Google Colab에서는

```
faiss-cpu
```

가 이미 설치된 경우가 많습니다.

그래서

설치 없이 사용

처럼 보입니다.

③ Chroma 같은 Vector DB

예

- Chroma

Chroma 내부

FAISS or HNSW

를 숨겨서 사용합니다.

그래서 사용자 입장에서는

FAISS 설치 안한 것처럼 보임

입니다.

12. 학생 실습 환경 (RTX 5070)

추천 스택

Gradio

Transformers

SentenceTransformers

FAISS

학생 PC에서 충분히 실행됩니다.

예

RTX 5070

16GB VRAM

가능한 모델

Mistral 7B
Llama 3 8B

✅ 핵심 정리

FAISS는

vector similarity search library

이며

`pip install faiss-cpu`

로 설치해야 합니다.

“설치 없이 가능”은 실제로는

wrapper library가 숨겨서 사용하는 경우

입니다.

원하시면 다음도 설명해 드리겠습니다.

“학생 **RTX 5070 PC**에서 돌아가는 **120줄**짜리 완전한 **RAG** 시스템 (**Gradio + FAISS + Mistral**)”

이 구조는 실제로 캡스톤 수업에서 가장 안정적으로 돌아가는 **RAG** 구조입니다.